As mentioned, the idea is that we can encapsulate all the business logic in a binary function that will compare current node in nearest neighbor search with the best solution found at a given stage. We as well assume that this predicate will also perform any necessary filtering, like for example making sure that current node's retailer has all items in stock. The changes needed to our search method are minimal, as shown in listing 11.7: we simply gather together filtering and comparison of current and best solution, and instead of just checking distances during our NN search, we use a function passed as an argument to `fNN`, function `cmpShops`[7], to decide which entry is "*nearer*".

Listing 11.8 A possible comparison method

```
function compareShops(boughtItems, (dist, shop), (nnDist,
nnShop))   #1
  if shop == null or not (item in shop.items " item in
boughtItems) then#2
    return 1
  else if nnShop == null
then                                                  #3
    return -1
  else if dist <= nnDist and heuristic(shop, nnShop)
then                      #4
    return -1
  else

#5
    return 1

fNN(root, customerPosition, compareShops([“Cheddar”, “Gravy”,
“Salad”])) #6
```